6

Advanced RAG Techniques for Information Retrieval and Augmentation

In the previous chapter, we discussed RAG and how this paradigm has evolved to solve some shortcomings of LLMs. However, even naïve RAG (the basic form of this paradigm) is not without its challenges and problems. Naïve RAG consists of a few simple components: an embedder, a vector database for retrieval, and an LLM for generation. As mentioned in the previous chapter, naïve RAG involves a collection of text being embedded in a database; once a query from a user arrives, text chunks that are relevant to the query are searched for and provided to the LLM to generate a response. These components allow us to respond effectively to user queries; but as we shall see, we can add additional components to improve the system.

In this chapter, we will see how in advanced RAG, we can modify or improve the various steps in the pipeline (data ingestion, indexing, retrieval, and generation). This solves some of the problems of naïve RAG and gives us more control over the whole process. We will later see how the demand for more flexibility led to a further step forward (modular RAG). We will also discuss important aspects of RAG, especially when the system (a RAG base product) is being produced. For example, we will discuss the challenges when we have a large amount of data or users. Also, since these systems may contain sensitive data, we will discuss both robustness and privacy. Finally, although RAG is a popular system today, it is still relatively new. So, there are still unanswered questions and exciting prospects for its future.

In this chapter, we'll be covering the following topics:

- Discussing naïve RAG issues
- Exploring advanced RAG pipelines
- Modular RAG and integration with other systems
- Implementing an advanced RAG pipeline
- Understanding the scalability and performance of RAG
- Open questions

Technical requirements

Most of the code in this chapter can be run on a CPU, but it is preferable for it to be run on a GPU. The code is written in PyTorch and uses standard libraries for the most part (PyTorch, Hugging Face Transformers, LangChain, **`chromadb`**, **`sentence-transformer`**, **`faiss-cpu`**, and so on).

The code for this chapter can be found on GitHub: https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr6.

Discussing naïve RAG issues

In the previous chapter, we introduced RAG in its basic version (called naïve RAG). Although the basic version of RAG has gone a long way in solving some of the most pressing problems of LLMs, several issues remain. For industrial applications, in particular (as well as medical, legal, and financial), naïve RAG is not enough, and we need a more sophisticated pipeline. We will now explore the problems associated with naïve RAG, each of which is associated with a specific step in the pipeline (query handling, retrieval, and generation).

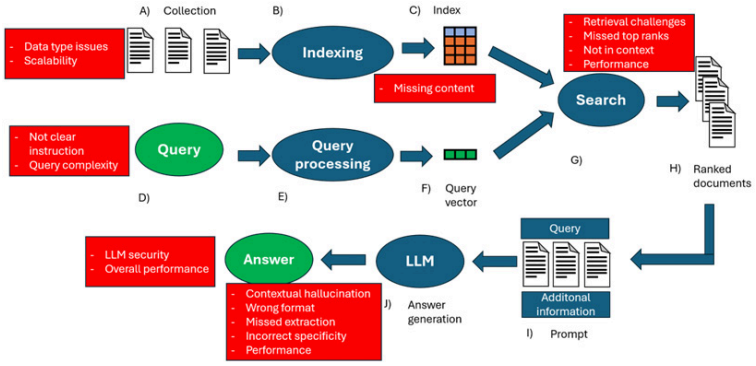

Figure 6.1 – Summary of naïve RAG issues and identifying different steps in the pipeline where the issues can arise

Let's discuss these issues in detail:

- **Retrieval challenges**: The phase of retrieval struggles with precision (retrieved chunks are misaligned) and recall (finding all relevant chunks). In addition, the knowledge base could be outdated. This could lead to either hallucinations or, depending on the prompt used, a response such as, "Sorry, I do not know the answer" or "The context does not allow the query to be answered." This can also be derived from poor database indexing or the documents being of different types (PDF, HTML, text, and so on) and being treated incorrectly (chunking for all file types is an example).
- **Missed top-rank documents**: Documents essential to answering the query may not be at the top of the list. By selecting the top *k* documents, we might select top chunks that are less relevant (or do not contain the answer) and not return the really relevant chunks to the LLM. The semantic representation capability of the embedding model may be weak (i.e., we chose an ineffective model because it was too small or not suitable for the domain of our documents).
- **Relevant information not in context**: Documents with the answer are found but there are too many to fit in the LLM's context. For example, the response might need several chunks, and these are too many for the context length of the model.
- **Failed extraction**: The right context might be returned to the LLM, but it might not extract the right answer. Usually, this happens when there is too much noise or conflicting information in the context. The model might generate hallucinations despite having the answer in the prompt (contextual hallucinations).

- **Answer in the wrong format**: There may be additional specifics in the query. For example, we may want an LLM to generate bullet points or report the information in a table. The LLM may ignore this information.
- **Incorrect specificity**: The generated answer is not specific enough or too specific with respect to the user's needs. This is generally a problem associated with how the system is designed and what its purpose is. Our RAG may be part of a product designed for students and must give clear and comprehensive answers on a topic. The model, on the other hand, might answer vaguely or too technically for a student. Typically, this is a problem when the query (or instructions) is not clear enough.
- **Augmentation hurdles or information redundancy**: Our database may contain information from different corpora, and many of the documents may contain redundant information or be in different styles and tones. The LLM could then generate repetition and/or create hallucinations. Also, the answer may not be good quality because the model fails to integrate the information from the various chunks.
- **Incomplete answers**: These are answers that are not wrong but are incomplete (this can result from either not finding all the necessary information or errors on the part of the LLM in using the context). Sometimes, it can also be a problem of the query being too complex ("Summarize items A, B, and C"), and so it might also be better to modify the query.
- **Lack of flexibility**: This when the system is not flexible; it does not currently allow efficient updating, and we cannot incorporate feedback from users, past interactions, and so on. The system does not allow us to handle certain files that are abundant in our corpus (for example, our system does not allow Excel).
- **Scalability and overall performance**: In this case, our system may be too slow to conduct an embedding, generate a response, and so on. Alternatively, we cannot handle embedding multiple documents per second, or we have performance issues that are specific to our product or domain. System security is a sore point, especially if we have sensitive data.

Now that we understand the issues with naïve RAG, let's understand how advanced RAG helps us tackle these issues.

Exploring the advanced RAG pipeline

Advanced RAG introduces a number of specific improvements to try to address the issues highlighted in naïve RAG. Advanced RAG, in other words, modifies the various components of RAG to try to optimize the RAG paradigm. These various modifications occur at the different steps of RAG: **pre-retrieval** and **post-retrieval**.

In the **pre-retrieval process**, the purpose is to optimize indexing and querying. For example, **adding metadata** enables more granular searching, and we provide more content to the LLM to generate text. Metadata can succinctly contain information that would otherwise be dispersed throughout the document.

In naïve RAG, we divide the document into different chunks and find the relevant chunks for each document. This approach has two limitations:

- When we have many documents, it impacts latency time and performance
- When the documents are large, we may not be able to easily find the relevant chunks

In naïve RAG, there is only one level (all chunks are equivalent even if they are derived from different documents). In general, though, for many corpora, there is a hierarchy, and it might be beneficial to use it.

To address these limitations, advanced RAG introduces several enhancements designed to improve both retrieval and generation. In the following subsections, we will explore some techniques

Hierarchical indexing

For a document consisting of several chapters, we could first find the chapters of interest and from there search for the various sections of interest. Since the chapters may be of considerable size (rich in noise), embedding may not best represent their contextual significance. The solution is to use summaries and metadata. In a **hierarchical index**, you create summaries at each hierarchical level (which can be considered abstracts). At the first level, we have summaries that highlight only the key points in large document segments. In the lower levels, the granularity will increase, and these abstracts will be closer and closer to only the relevant section of data. Next, we will conduct the embedding of these abstracts. At inference time, we will calculate the similarity with these summary embeddings. Of course, this means either we manually write the summaries or we use an LLM to conduct summarization. Then, using the associated metadata, we can find the chunks that match the summary and provide it to the model.

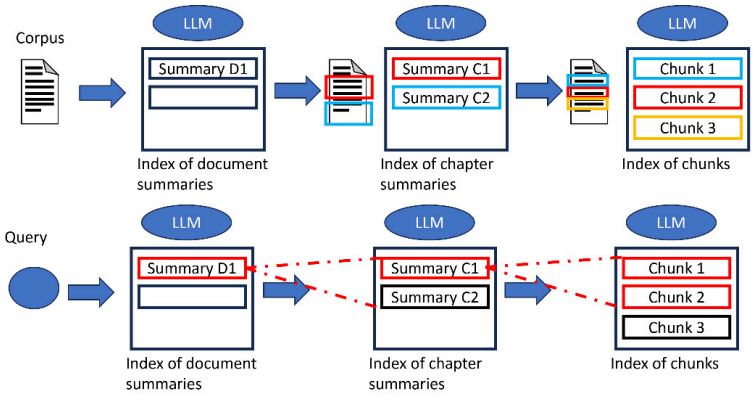

Figure 6.2 – Hierarchical indexing

As seen in the preceding figure, the corpus is divided into documents; we then obtain a summary of each document and embed it (in naïve RAG, we were dividing into chunks and embedding the chunks). In the next step, we embed the summary of a lower hierarchical level of the documents (chapter, sections, heading, and subheadings) until we reach the chunk level. At inference time, a similarity search is conducted on the summaries to retrieve the chunks we are interested in.

There are some variations to this approach. For more control, we can choose a split approach for each file type (HTML, PDF, and GitHub reposi-

tory). In this way, we can make the summary data type-specific and embed the summary, which works as a kind of text normalization.

When we have documents that are too long for our LLM summarizer, we can use **map and reduce**, where we first conduct a summarization of various parts of the document, then collate these summaries and get a single summary. If the documents are too encyclopedic (i.e., deal with too many topics), there is a risk of semantic noise impacting retrieval. To solve this, we can have multiple summaries per document (e.g., one summary per 10K tokens or every 10 pages of document).

Hierarchical indexing improves the contextual understanding of the document (because it respects its hierarchy and captures the relationships between various sections, such as chapters, headings, and subheadings). This approach allows greater accuracy in finding the results, and they are more relevant. On the other hand, this approach comes at a cost both during the pre-retrieval stage and in inference. Too many levels and you risk having a combinatorial explosion, that is, a rapid growth in complexity due to the exponential increase in possible combinations, with a huge latency cost.

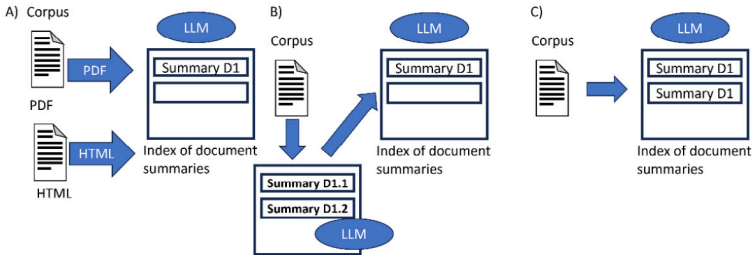

Figure 6.3 – Hierarchical indexing variation

In the preceding figure, we can see these hierarchical index variations:

- *A*: Different handling for each document type to better represent their structure
- *B*: Map and reduce to handle too-long documents (intermediate summaries are created and then used to create the final document summary)
- *C*: Multi-summary for each document when documents are discussing too many topics

Hypothetical questions and HyDE

Another modification of the naïve RAG pipeline is to try to make chunks and possible questions more semantically similar. By having an idea of who our users are, we can imagine the kind of use they will get out of our system (for a chatbot, most queries will be questions, so we can tailor the system toward these kinds of queries). **Hypothetical questions** is a type of strategy in which we use an LLM to generate one (or more) hypothetical question(s) for each chunk. These hypothetical questions are then transformed into vectors (embedding), and these vectors are used to do a similarity search when there is a query from a user. Of course, once we have identified the hypothetical questions most similar to our real query, we find the chunks (thanks to the metadata) and provide them to the model. We can generate either a single query or multiple queries for each chunk (this increases the accuracy as well as the computational cost). In this case, we are not using the vector representation of chunks (we do not

conduct embeddings of chunks but hypothetical questions). Also, we do not necessarily have to save the hypothetical questions, just their vectors (the important thing is that we can map them back to the chunks).

Hypothetical Document Embeddings (**HyDE**) instead tries to convert the user answers to better match the chunks. Given a query, we create hypothetical answers to it. After that, we conduct embeddings of these generated answers and carry out a similarity search to find the chunks of interest. These generated answers should be most semantically similar to the user's query, allowing us to be able to find better chunks. In some variants, we create five different generated answers and conduct the average of their embedding vectors before conducting the similarity search. This approach can help when we have a low recall metric in the retrieval step or when the documents (or queries) come from a specific domain that is different from the retrieval domain. In fact, embedding models generalize poorly to knowledge domains that they have not seen. An interesting little note is that when an LLM generates these hypothetical answers, it does not know the exact answer (that is not even the purpose of the approach) but is able to capture relevant patterns in the question. We can then use these captured patterns to retrieve the chunks.

Let's look in detail at the difference between the two approaches. With the hypothetical questions approach, we generate hypothetical questions and use the embedding of these hypothetical questions to then find the chunks of interest. With HyDE, we generate hypothetical answers to our query and then use the embedding of these answers to find the chunks of interest.

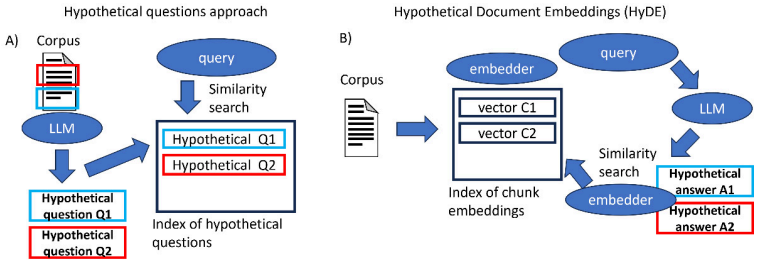

Figure 6.4 – Hypothetical questions and HyDE approaches

We can then look in detail at the differences between the two approaches, imagining that we have a hypothetical user question ("What are the potential side effects of using acetaminophen?"):

- **Pre-retrieval phase**: During this phase, we have to create our drug embedding database. We reduce our documents (sections of a drug's safety report) into chunks. In the hypothetical questions approach, for each chunk, hypothetical questions are generated using an LLM (for example, "What are the side effects of this drug?" or "Are there any adverse reactions mentioned?"). Each of these hypothetical questions is then embedded into a vector space (a database of vectors for these questions). At this stage, HyDE is equal to classic RAG; no variation is conducted.
- **Query phase**: In the hypothetical questions approach, when a user submits the query, it is embedded and matched against the embedded hypothetical questions. The system looks for the hypothetical questions that are most similar to the user's question (in this case, it might be, "What are the side effects of this drug?"). At this point, the chunks from which these hypothetical questions were generated are identi-

fied (we use metadata). These chunks are provided in context for generation. In HyDE, when the user query arrives, an LLM generates hypothetical answers (for example, “Paracetamol may cause side effects such as nausea, liver damage, and rashes” or “Potential adverse reactions include dizziness and gastrointestinal discomfort”).
Note that these answers are generated using LLM knowledge without retrieval. At this point, we conduct the embedding of these hypothetical answers (we use an embedding model), then conduct the embedding of the query and try to match it with the embedded hypothetical answers. For example, “Paracetamol may cause side effects such as nausea, liver damage, and rashes” is the one closest to the user query. We then search for the chunks closest to these hypothetical answers and provide the LLM to generate the context.

Context enrichment

Another technique is **context enrichment**, in which we find smaller chunks (greater granularity for better search quality) and then add surrounding context. **Sentence window retrieval** is one such technique in which each sentence in a document is embedded separately (the embedded textual unit is smaller and therefore more granular). This allows us to have higher precision in finding answers, though we risk losing context for LLM reasoning (and thus worse generation). To solve this, we expand our context window. Having found a sentence *x*, we take *k* sentences that surround it in the document (sentences that are before and after our sentence *x* in the document).

Parent document retriever is a similar technique that tries to find a balance between searching on small chunks and providing context with larger chunks. The documents are divided into small child chunks, but we preserve the hierarchy of their parent documents. In this case, we conduct embedding of small chunks that directly address the specifics of a query (ensuring larger chunks’ relevance). But then we find the larger parent documents (to which the found chunks belong) and provide them to the LLM for generation (more contextual information and depth). To avoid retrieving too many parent documents, once the top *k* chunks are found, if more than *n* chunks belong to a parent document, we add this document to the LLM context.

These approaches are depicted in the following figure:

1. Once a chunk is found, we expand the selection with the previous and next chunks.
2. We conduct embedding of small chunks and find the top *k* chunks; if most chunks (greater than a parameter *n*) are derived from a document, we provide the LLM with the document as the context.

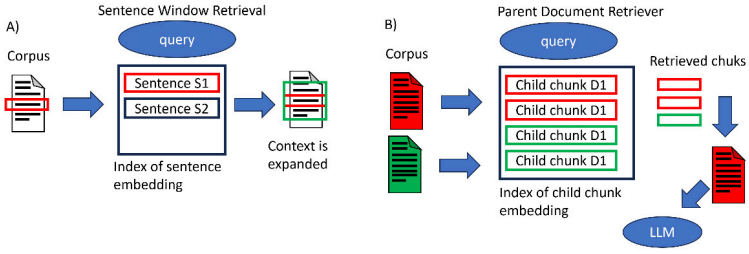

Figure 6.5 – Context enrichment approaches

Query transformation

Query transformation is a family of techniques that leverages an LLM to improve retrieval. If a query is too complex, it can be decomposed into a series of queries. In fact, we may not find a chunk that responds to the query, but more easily find chunks that respond to each subquery (e.g., "Who was the inventor of the telegraph and the telephone?" is best broken down into two independent queries). **Step-back prompting** uses an LLM to generate a more general query that can match a high-level context. It stems from the idea that when a human being is faced with a difficult task, they take a step back and do abstractions to get to the high-level principles. In this case, we use the embedding of this high-level query and the user's query, and both found contexts are provided to the LLM for generation. **Query rewriting**, on the other hand, reformulates the initial query with an LLM to make retrieval easier.

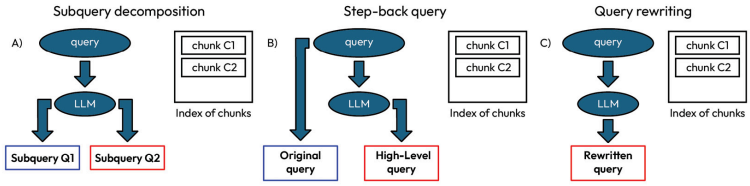

Figure 6.6 – Three examples of query transformation

Query expansion is a technique similar to query rewriting. Underlying it is the idea that adding terms to the query can allow it to find relevant documents that do not have lexical overlap with the query (and thus improve retrieval recall). Again, we use an LLM to modify the query. There are two main possibilities:

- Ask an LLM to generate an answer to the query, after which the generated answer and the query are embedded and used for retrieval.
- Generate several queries similar to the original query (usually a prefixed number *n*). This *n* set of queries is then vectorized and used for search.

This approach usually improves retrieval because it helps disambiguate the query and find documents that otherwise would not be found; it also helps the system better compile the query. On the other hand, though, it also leads to finding irrelevant documents, so it pays to combine it with post-processing techniques for finding documents.

Keyword-based search and hybrid search

Another way to improve search is to focus not only on contextual information but also on keywords. **Keyword-based search** is a search by an exact match of certain keywords. This type of search is beneficial for specific terms (such as product or company names or specific industry jargon). However, it is sensitive to typos and synonyms and does not capture context. **Vector or semantic search**, on the contrary, finds the semantic meaning of a query but does not find exact terms or keywords (which is sometimes essential for some queries, especially in some domains such as marketing). **Hybrid search** takes the best of both worlds by combining a model for keyword search and vectorial search.

The most commonly used model for keyword search is BM25 (which we discussed in the previous chapter), which generates sparse embeddings.

BM25 then allows us to identify documents that contain specific terms in the query. So, we create two embeddings: a sparse embedding with BM25 and a dense embedding with a transformer. To select the best chunks, you generally try to balance the impact of your different types of searches. The final score is a weighted combination (you use an alpha hyperparameter) of the two scores:

$$score_{hybrid} = 1 - \alpha) \bullet score_{sparse} + \alpha \bullet score_{dense}$$

α has a value between 0 and 1 (0 means pure vectorial search, while 1 means only keyword search). Typically, the value of α is 0.4 or 0.5 (other articles even suggest 0.3).

As a practical example, we can imagine an e-commerce platform with a vast product catalog containing millions of items across categories such as electronics, fashion, and home appliances. Users search for products with different types of queries, which may include the following:

- Specific terms such as a brand or product name (e.g., "iPhone 16")
- A general description (e.g., "Medium-price phone with a good camera")
- Queries that contain mixed elements (e.g., "iPhone with cost less than $500")

A pure keyword-based search (such as the BM25 algorithm) would struggle with vague or purely descriptive descriptions, while a vector-based semantic search might miss exact matches for a product. Hybrid search combines the best of both. BM25 prioritizes exact matches, such as matches of "iPhone," allowing us to find specific items using keywords. Semantic search allows us to capture the semantic meaning of phrases such as "phone with a good camera." Hybrid search is a great solution for all three of the previously mentioned cases.

Query routing

So far, we have assumed that once a query arrives, it is used for a search within the vector database. In reality, we may want to conduct the search differently or control the flow within the system. For example, the system should be able to interact with different types of databases (vector, SQL, and proprietary databases), different sources, or different types of modalities (image, text, and sound). Some queries do not, then, need to be searched with RAG; the parametric memory of the model might suffice (we will discuss this in more depth in the *Open questions and future perspectives* section). Query routing thus allows control over how the system should respond to the query. You can imagine it as being a series of if/else causes, though instead of being hardcoded, we have a router (usually an LLM) that makes a decision whenever a query arrives. Obviously, this means that we have a nondeterministic system, and it will not always make the right decision, although it can have a major positive impact on performance.

The router can be a set of logical rules or a neural model. Some options for a router are the following:

- **Logical routers**: A set of logical rules that can be if/else clauses (e.g., if the query is an image, it searches the image database; otherwise, it

searches the text database). Logical routers don't understand the query, but they are very fast and deterministic.

- **Keyword routers**: A slightly more sophisticated alternative in which we try to select a route by matching keywords between the query and a list of options. This search can be done with a sparse encoder, a specialized package, or even an LLM.
- **Zero-shot classification router**: Zero-shot classification is a task in which an LLM is asked to classify an item with a set of labels without being specified and trained for it. Each query is given to an LLM that must assign a route label from those in a list.
- **LLM function calling router**: The different routes are described as functions (with a specific description) and the model must decide where to direct the queries by selecting the function (in this approach, we leverage its decision-making ability).
- **Semantic router**: In this approach, we use a semantic search to decide on the best route. In short, we have a list of example queries and the associated routes. These are then embedded and saved as vectors in a database. When a query arrives, we conduct a similarity search with the other queries in our database. We then select the option associated with the query with the best similarity match.

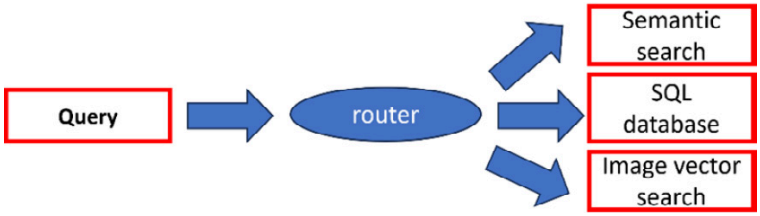

Figure 6.7 – Query routing

Once we have found the context, we need to integrate it with the query and provide it to the LLM for generation. There are several strategies to improve this process, usually called **post-retrieval strategies**. After the vector search, retrieval returns the top *k* documents (an arbitrary cutoff that is determined in advance). This can lead to the loss of relevant information. The simplest solution is to increase the value of the top *k* chunks. Obviously, we cannot return all retrieved chunks, both because they would not fit into the context length of the model and because the LLM would then have problems with handling all this information (efficient use of a long context length).

We can imagine a company offering different services across different domains, such as banking, insurance, and finance. Customers interact with a chatbot to seek assistance with banking services (account details, transactions, and so on), insurance services (policy details, claims, etc.), and financial services (suggestions, investments, etc.). Each domain is different. Due to regulations and privacy issues, we want to prevent a chatbot from searching for details for a customer of another service. Also, searching all databases for every query is inefficient and leads to more latency and irrelevant results.

Reranking

One proposed solution to this dilemma is to maximize document retrieval (increase the top *k* retrieved results and thus increase the retrieval recall metric) but at the same time maximize the LLM recall (by minimizing the

number of documents supplied to the LLM). This strategy is called **reranking**. Reranking consists of two steps:

1. First, we conduct a classical retrieval and find a large number of chunks.
2. Next, we use a reranker (a second model) to reorder the chunks and then select the top *k* chunks to provide to the LLM.

The reranker improves the quality of chunks returned to the LLM and reduces hallucinations in the system. In addition, reranking considers contrasting information (related to the query) and then considers chunks in context with the query. There are several types of rerankers, each with its own limitations and advantages:

- **Cross-encoders**: These are transformers (such as BGE) that take two textual sequences (the query and the various chunks one at a time) as input and return the similarity between 0 and 1.
- **Multi-vector rerankers**: These are still transformers (such as ColBERT) and require less computation than cross-encoders (the interaction between the two sequences is late-stage). The principle is similar; given two sequences, they return a similarity between 0 and 1. There are improved versions with a large context length, such as jina-colbert-v1-en.
- **LLMs for reranking**: LLMs can also be used as rerankers. Several strategies are used to improve the ranking capabilities of an LLM:
  - **Pointwise methods** are used to calculate the relevance of a query and a single document (also referred to as zero-shot document reranking).
  - **Pairwise methods** consist of providing an LLM with both the query and two documents and asking it to choose which one is more relevant.
  - **Listwise methods**, on the other hand, propose to provide a query and a list of documents to the LLM and instruct it to produce as output a ranked list. Models such as GPT are usually used, with the risk of high computational or economic costs.
- **Fine-tuned LLMs**: This is a class of models that is specifically for ranking tasks. Although LLMs are generalist models, they do not have specific training for ranking and therefore cannot accurately measure query-document relevance. Fine-tuning allows them to improve their capability. Generally, there are two types of models used: encoder-decoder transformers (RankT5) or decoder-only transformers (e.g., derivatives of Llama and GPT).

All these approaches have an impact on both performance (retrieval quality) and cost (computational cost, system latency, and potential system cost). Generally, multi-vectors are those with lower computational cost and discrete performance. LLM-based methods may have the best performance but have high computational costs. In general, reranking has a positive impact on the system, which is why it is often a component of the pipeline.

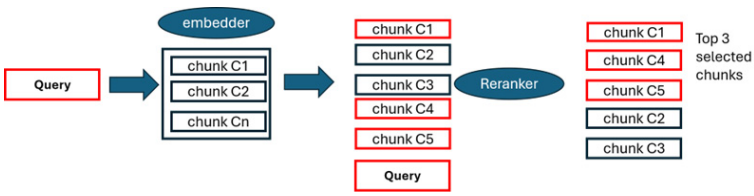

Figure 6.8 – Reranking approach. Chunks highlighted in red are the chunks relevant to the query

Alternatively, there are other **post-processing techniques**. For example, it is possible to filter out chunks if the similarity achieved is below a certain score threshold, if they do not include certain keywords, if a certain value is not present in the metadata associated with the chunks, if the chunks are older than a certain date, and many other possibilities. An additional strategy is that once we have found chunks, starting from the embedding vectors, we conduct **k-nearest neighbors (kNN)** research. In other words, we add other chunks that are neighbors in the latent space of those found (this strategy can be done before or after reranking).

In addition, once the chunks are selected to be provided in context to the LLM, we can alter their order. As shown in the following figure, a study published in 2023 shows that the best performance for an LLM is when the important information is placed at the beginning or end of the input context length (performance drops if the information is in the middle of the context length, especially if it is very long):

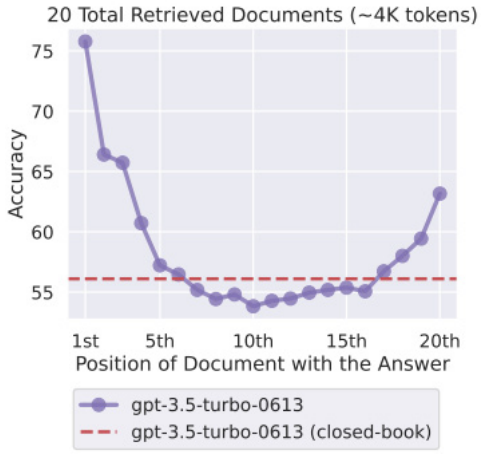

Figure 6.9 – Changing the location of relevant information impacts the performance of an LLM (**https://arxiv.org/abs/2307.03172**)

That is why it has been proposed to **reorder the chunks**. They can be placed in order of relevance, but also in alternating patterns (chunks with an even index are placed at the beginning of the list and chunks with an odd index at the end). The alternating pattern is used especially when using wide top *k* chunks, so the most relevant chunks are placed at the beginning and end (while the less relevant ones are in the middle of the context length).

You can notice that reranking improves the performance of the system:

Reranker model	Avg.	NQ	HotpotQA	FiQA
***Embedding:* snowflake-arctic-embed-l**	0.6100	0.6311	0.7518	0.4471
+ ms-marco-MiniLM-L-12-v2	0.5771	0.5876	0.7586	0.3850
+ mxbai-rerank-large-v1	0.6077	0.6433	0.7401	0.4396
+ jina-reranker-v2-base-multilingual	0.6481	0.6768	0.8165	0.4511
+ bge-reranker-v2-m3	0.6585	0.6965	0.8458	0.4332
+ NV-RerankQA-Mistral-4B-v3	**0.7529**	**0.7788**	**0.8726**	**0.6073**
***Embedding:* NV-EmbedQA-e5-v5**	0.6083	0.6380	0.7160	0.4710
+ ms-marco-MiniLM-L-12-v2	0.5785	0.5909	0.7458	0.3988
+ mxbai-rerank-large-v1	0.6077	0.6450	0.7279	0.4502
+ jina-reranker-v2-base-multilingual	0.6454	0.6780	0.7996	0.4585
+ bge-reranker-v2-m3	0.6584	0.6974	0.8272	0.4506
+ NV-RerankQA-Mistral-4B-v3	**0.7486**	**0.7785**	**0.8470**	**0.6203**
***Embedding:* NV-EmbedQA-Mistral7B-v2**	0.7173	0.7216	0.8109	0.6194
+ ms-marco-MiniLM-L-12-v2	0.5875	0.5945	0.7641	0.4039
+ mxbai-rerank-large-v1	0.6133	0.6439	0.7436	0.4523
+ jina-reranker-v2-base-multilingual	0.6590	0.6819	0.8262	0.4689
+ bge-reranker-v2-m3	0.6734	0.7028	0.8635	0.4539
+ NV-RerankQA-Mistral-4B-v3	**0.7694**	**0.7830**	**0.8904**	**0.6350**

Figure 6.10 – Reranking improves the performance in question-answering (https://arxiv.org/pdf/2409.07691)

In addition to reranking, several complementary techniques can be applied after the retrieval stage to further refine the information passed to the LLM. These include methods for improving citation accuracy, managing chat history, compressing context, and optimizing prompt formulation. Let's have a look at some of them.

Reference citations is not really a technique for system improvement, but it is highly recommended as a component of a RAG system. Especially if we are using different sources to compose our query response, it is good to keep track of which sources were used (e.g., the documents that the LLM used). We can simply safeguard the sources that were used for generation (which documents the chunks correspond to). Another possibility is to mention in the prompt for the LLM the sources used. A more sophisticated technique is fuzzy citation query engine. Fuzzy matching is a string search to match the generated response to the found chunks (a technique that is based on dividing the words in the chunk into n-grams and then conducting a TF-IDF).

ChatEngine is another extension of RAG. Conducting fine-tuning of the model is complex, but at the same time, we want the LLM to remember previous interactions with the user. RAG makes it easy to do this, so we can save previous dialogues with users. A simple technique is to include the previous chat in the prompt. Alternatively, we can conduct embedding of the chats and find the highlights. Another technique is to try to capture the context of the user dialogue (chat logic). Since the discussion can wind through several messages, one solution to avoid a prompt that may exceed the context length is **prompt compression**. We reduce the prompt length by reducing the previous interaction with the user.

In general, **contextual compression** is a concept that helps the LLM during generation. It also saves computational (or economic, if using a model via an API) resources. Once the documents are found, we can compress the context, with the aim of retaining only the relevant information. In fact, the context often also contains information irrelevant to the query, or even repetitions. Additionally, most of the words in a sentence could be predicted directly from the context and are not needed to provide the information to the LLM during generation. There are several strategies to reduce the prompt provided to the LLM:

- **Context filtering**: In information theory, tokens with low entropy are easily predictable and thus contain redundant information (provide

less relevant information to the LLM and have little impact on its understanding of the context). We therefore use an LLM that assigns an information value to each lexical unit (how much it expects to see that token or sentence in context). We conduct a ranking in descending order and keep only those tokens that are in the first p-th percentile (we decide this *a priori*, or it can be context-dependent).

- **LongLLMLingua**: This is another approach based on information entropy and using information from both context and query (question aware). The approach conducts dynamic compression and reordering of documents to make generation more efficient.
- **Autocompressors**: This uses a kind of fine-tuning of the system and summary vectors. The idea behind it is that a long text can be summarized in a small vector representation (summary vectors). These vectors can be used as soft prompts to give context to the model. The process relies on keeping the LLM's weights frozen while introducing trainable tokens into the prompt. These tokens are learned during training, enabling the system to be optimized end-to-end without modifying the model's core parameters. During generation, these vectors are joined, and the model is then context-aware. Already trained models exist, as follows:

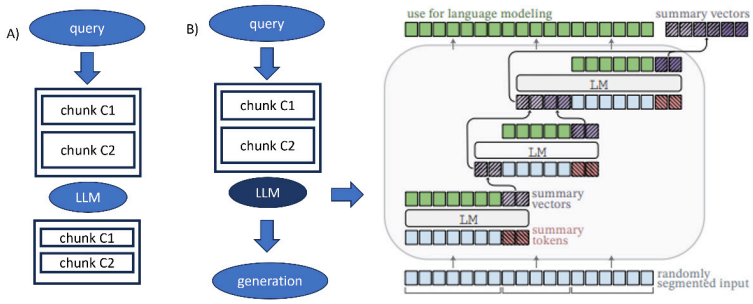

Figure 6.11 – A) Context compression and filtering. B) Autocompressor. (Adapted from https://arxiv.org/abs/2305.14788)

Prompt engineering is another solution to improve generation. Some suggestions are common to any interaction with an LLM. Thus, principles such as providing clear ("Reply using the context") and unambiguous ("If the answer is not in the context, write I do not know") instructions apply to RAG. There may, however, be specific directions or even examples for designing the best possible prompt for our system. Other instructions may be specific to how we want the output (for example, as a list, in HTML, and so on). There are also libraries for creating prompts for RAG that follow a specific format.

Response optimization

The last step in a pipeline before conducting the final response is to improve the response from the user. One strategy is that of the **response synthesizer**. The basic strategy is to concatenate the prompt, context, and query and provide it to the LLM for generation. More sophisticated strategies involve more calls from the LLM. There are several alternatives to this idea:

- Iteratively refine the response using one chunk at a time. The previous response and a subsequent chunk are sent to the model to improve the response with the new information.
- Generate several responses with different chunks, then concatenate them all together and generate a summary response.

- Hierarchical summarization starts with the responses generated for each different context and recursively combines them until we arrive at a single response. While this approach enhances the quality of both summaries and generated answers, it requires significantly more LLM calls, making it costly in terms of both computational resources and financial expense.

An interesting development is the possibility of using RAG as a component of an agent system. As we introduced in ***Chapter 4***, RAG can act as the memory of the system. RAG can be combined with **agents**. An LLM is capable of reasoning that can be merged with RAG and call-up tools or connect to sites when a query requires additional steps. An agent can also handle different components (retrieve chat history, conduct query routing, connect to APIs, and execute code). A complex RAG pipeline can have several components that are not the best fit for every situation, and an LLM can decide which are the best components to use.

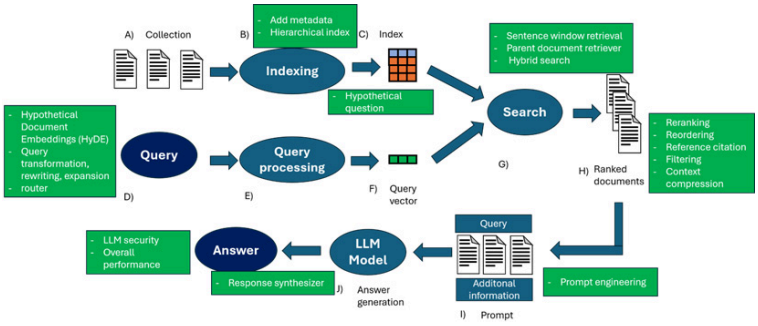

Figure 6.12 – Different elements in a pipeline of advanced RAG

So far, we have assumed that a pipeline should be executed only once. The standard practice is we conduct retrieval once and then generate. This approach, though, can be insufficient for complex problems that require multi-step reasoning. There are three possibilities in this case:

- **Iterative retrieval**: In this case, the retrieval is conducted multiple times. Given a query, we conduct the retrieval, we generate the result, and then the result is judged by an LLM. Depending on the judgment, we repeat the process up to *n* times. This process improves the robustness of the answers after each iteration, but it can also lead to the accumulation of irrelevant information.
- **Recursive retrieval**: This system was developed to increase the depth and relevance of search results. It is similar to the previous one, but at each iteration, the query is refined in response to previous search results. The purpose is to find the most relevant information by exploiting a feedback loop. Many of these approaches exploit **chain-of-thought** (**CoT**) to guide the retrieval process. In this case, the system then breaks down the query into a series of intermediate steps that it must solve. This approach is advantageous when the query is not particularly clear or when the information sought is highly specialized or requires careful consideration of nuanced details.
- **Adaptive retrieval**: In this case, the LLM actively determines when to search and whether the retrieved content is optimal. The LLM judges not only the retrieval step but also its own operation. The LLM can decide when to respond, when to search, or whether additional tools are needed. This approach is often used not only when searching on the RAG but also when conducting web searches. Flare (an adaptative approach to RAG) analyzes confidence during the generation process and

makes a decision when the confidence falls below a certain threshold. Self-RAG, on the other hand, introduces **reflection tokens** to monitor the process and force an introspection of the LLM.

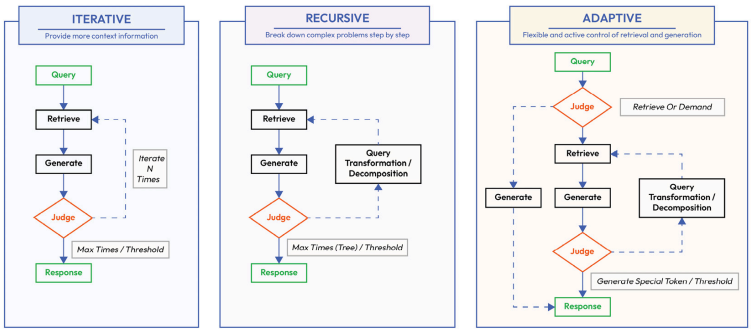

Figure 6.13 – Augmentation of RAG pipelines (https://arxiv.org/pdf/2312.10997)

To better understand how advanced RAG techniques address known limitations, *Table 6.1* presents the mapping between key problems and the most effective solutions proposed in recent research.

Problem to Solve	Solution
Issues in naïve RAG: Latency and performance degradation with many or large documents	Use hierarchical indexing: Summarize large sections, create multi-level embeddings, use metadata, and implement variations such as map-reduce for long documents or multi-summary for diverse topics.
Flat hierarchy limits relevance when the corpus contains an inherent structure	Apply hierarchical indexing: Respect the document's structure (chapters, headings, and subheadings), and retrieve context based on hierarchical summaries and embeddings.
Low retrieval accuracy and domain-specific generalization challenges	Generate and embed hypothetical questions for each chunk (Hypothetical Qs). Use HyDE: generate hypothetical answers to match query semantics, embed them, and retrieve relevant chunks.
Loss of context in granular chunking	Use context enrichment: Expand retrieved chunks with surrounding context using sentence windows or retrieve parent documents to broaden context.
Complex queries and low recall from initial retrieval	Apply query transformation: Decompose complex queries into subqueries, use step-back prompting or query expansion. Embed transformed queries for improved retrieval.
Context mismatch for specific terms or keywords	Use hybrid search: Combine keyword-based (e.g., BM25) and vector-based retrieval using weighted scoring.

Inefficiency in managing diverse query types	Implement query routing: Use logical rules, keyword-based or semantic classifiers, zero-shot models, or LLM-based routers to direct queries to the appropriate backends.
Loss of relevant chunks due to arbitrary top-k cutoff	Apply reranking: Use cross-encoders, multi-vector rerankers, or LLM-based (pointwise, pairwise, or listwise) reranking to reorder retrieved chunks.
Loss of information or efficiency in LLM context	Use context compression: Filter low-entropy tokens, compress or reorder chunks dynamically (e.g., LongLLMLingua), or apply summary vectors and autocompressors.
Inefficient response generation	Optimize responses: Use iterative refinement, hierarchical summarization, or multi-step response synthesis. Improve prompt quality and specificity.
Memory limitations in dialogue systems	Use ChatEngine techniques: Save and embed past conversations, compress user dialogue, and merge chat history with current queries.
Need for complex reasoning or dynamic query adaptation	Adopt adaptive and multi-step retrieval: Use recursive, iterative approaches with feedback loops and self-reflection (e.g., Flare, Self-RAG).
Lack of source tracking in generated responses	Include citations: Use fuzzy citation matching, metadata tagging, or embed source references in prompts.
Need for pipeline customization based on query complexity or modality	Augment RAG pipelines: Combine with agents for reasoning, tool use, and decision-making. Apply adaptive and recursive retrieval loops for complex queries.

Table 6.1 – Problems and solutions in RAG

Modular RAG and its integration with other systems

Modular RAG is a further advancement; it can be considered as an extension of advanced RAG but focused on adaptability and versatility. In this sense, the modular system means it has separate components that can be used either sequentially or in parallel.

The pipeline itself is remodeled, with alternating search and generation. In general, modular RAG involves optimizing the system toward performance and adapting to different tasks. Modular RAG introduces modules

for this that are specialized. Some examples of the modules that are included are as follows:

- **Search module**: This module is responsible for finding relevant information about a query. It allows searching through search engines, databases, and **knowledge graphs** (**KGs**). It can also use sophisticated search algorithms, use machine learning, and execute code.
- **Memory module**: This module serves to store relevant information during the search process. In addition, the system can retrieve context that was previously searched.
- **Routing module**: This module tries to identify the best path for a query, where it can either search for different information in different databases or decompose the query.
- **Generation module**: Different queries may require a different type of generation, such as summarization, paraphrasing, and context expansion. The focus of this module is on improving the quality and relevance of the output.
- **Task-adaptable module**: This module allows dynamic adaptation to tasks that are requested from the system. In this way, the system dynamically adjusts retrieval, processing, and generation.
- **Validation module**: This module evaluates retrieved responses and context. The system can identify errors, biases, and inconsistencies. The process becomes iterative, in which the system can improve its responses.

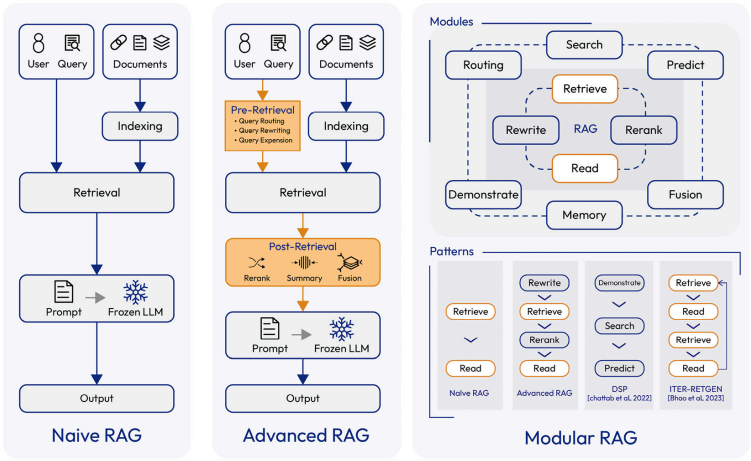

Figure 6.14 – Three different paradigms of RAG (https://arxiv.org/pdf/2312.10997)

Modular RAG offers the advantage of adaptability because these modules can be replaced or reconfigured as needed. The flow between different modules can be finely tuned, allowing an additional level of flexibility. Furthermore, if naïve and advanced RAG are characterized by a "retrieve and read" mechanism, modular RAG allows "retrieve, read, and rewrite." In fact, through the ability to evaluate and provide feedback, the system can refine the response to the query.

As this new paradigm spread, interesting alternatives were experimented with, such as integrating information coming from the parametric memory of the LLM. In this case, the model is asked to generate a response before retrieval (recite and answer). **Demonstrate-search-predict** (**DSP**) shows how you can have different interactions between the LLM and RAG to solve complex queries (or knowledge-intensive tasks). DSP shows how a modular RAG allows for robust and flexible pipelines at the same time. **Self-reflective retrieval-augmented generation** (**Self-RAG**), on the

other hand, introduces an element of criticism into the system. The LLM reflects on what it generates, critiquing its output in terms of factuality and overall quality. Another alternative is to use interleaved CoT generation and retrieval. These approaches usually work best when we have issues that require reasoning.

Training and training-free approaches

RAG approaches fall into two groups: training-free and training-based. Naïve RAG approaches are generally considered training-free. **Training-free** means that the two main components of the system (the embedder and LLM) are kept frozen from the beginning. This is possible because they are two components that are pre-trained and therefore have already acquired capabilities that allow us to use them.

Alternatively, we can have three types of **training-based approaches**: independent training, sequential training, and joint training.

In **independent training**, both the retriever and LLMs are trained separately in totally independent processes (there is no interaction during training). In this case, we have separate fine-tuning of the various components of the system. This approach is useful when we want to adapt our system to a specific domain (legal, financial, or medical, for example). Compared to a training-free approach, this type of training improves the capabilities of the system for the domain of our application. LLMs can also be fine-tuned to make better use of the context.

Sequential training, on the otherRAG:sequential training" hand, assumes that we use these two components sequentially, so it is better to find a form of training that increases the synergy between these components. The components can first be trained independently, following which they are trained sequentially. One of the components is kept frozen while the other undergoes additional training. Depending on what the order of training is, we can have two classes, retriever-first or LLM-first:

- **Retriever-first**: In this class, the trainer's training is conducted and then it is kept frozen. Then, the LLM is trained to understand how to use the knowledge in the retriever context. For example, we conduct the fine-tuning of our retriever independently and then we conduct fine-tuning of the LLM using the retrieved chunks. The LLM receives the retriever chunks during its fine-tuning and learns how best to use this context for generation.
- **LLM-first**: This is a bit more complex, but it uses the supervision of an LLM to train the retriever. An LLM is usually a much more capable model than the retriever because it has many more parameters and has been trained on many more tokens, thus making it a good supervisor. In a sense, this approach can be seen as a kind of knowledge distillation in which we take advantage of the greater knowledge of a larger model to train a smaller model.

Training Approach	Domains/Applications	Reasoning
Retriever-first	**Search engines (general or domain-specific)**	Focuses on retrieving the most relevant documents quickly and accu-

	For example, legal document search, medical literature search, or e-commerce product search	rately. Essential for systems where domain-specific precision is critical, and the retriever must handle vast, structured, or semi-structured corpora.
	Enterprise knowledge management For example, internal corporate documentation, FAQs, or CRM systems	Emphasizes retrieving the right documents efficiently from proprietary databases, where the quality of retrieval has a more significant impact than the quality of generation.
	Scientific research repositories For example, PubMed, arXiv, or patents	Ensures precise and recall-optimized retrieval in highly technical or specialized fields where high-quality retrieval is essential for downstream tasks such as summarization or report generation.
	Regulatory and compliance systems For example, financial compliance checks, or legal case law databases	In domains where accuracy and compliance are critical, the retriever must reliably surface the most relevant content while minimizing irrelevant or low-confidence retrievals.
LLM-first	**Conversational agents** For example, customer support chatbots or personal assistants	Relies heavily on the generative capabilities of the LLM to provide nuanced, conversational responses. Retrieval is secondary as the LLM interprets and integrates retrieved content.
	Creative applications For example, content writing, storytelling, or brainstorming	The LLM's ability to create, synthesize, and infer from retrieved data is paramount. Retrieval supports generation by providing a broader context rather than being the focal point of optimization.

	Complex reasoning tasks For example, multi-step problem-solving or decision-making systems	The LLM's role as a reasoner outweighs retrieval precision, as the focus is on the ability to process, relate, and infer knowledge. Retrieval primarily ensures access to supplementary information for reasoning.
	Educational tools For example, learning assistants or personalized tutoring systems	The LLM's ability to adapt and generate instructional content tailored to the user's context is more critical than precise retrieval. Retrieval serves as a secondary mechanism to ensure the completeness of information.

Table 6.2 – Training approaches

According to this article (Izacard, https://arxiv.org/abs/2012.04584), attention activation values in the LLM are a good proxy for defining the relevance of a document, so they can be used to provide a label (a kind of guide) to the retriever on how good the search results are. Hence, the retriever is trained with a metric based on attention in the LLM. For a less expensive approach, a small LLM can be used to generate the label to then train the retriever. There are then variations in these approaches, but all are based on the principle that once we have fine-tuned the LLM, we want to align the retriever.

Joint methods, on the other hand, represent end-to-end training of the system. In other words, both the retriever and the generator are aligned at the same time (simultaneously). The idea is that we want the system to simultaneously improve both its ability to find knowledge and its ability to use this knowledge for generation. The advantage is that we have a synergistic effect during training.

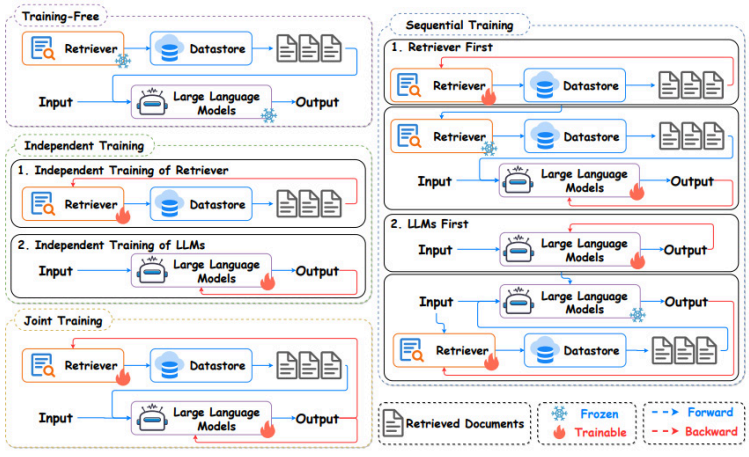

Figure 6.15 – Different training methods in RAG (https://arxiv.org/pdf/2405.06211)

Now that we know the different modifications that we can apply to our RAG, let's try them in the next section.

Implementing an advanced RAG pipeline

In this section, we will describe how an advanced RAG pipeline can be implemented. In this pipeline, we use a more advanced version of naïve RAG, including some add-ons to improve it. This shows us how the starting basis is a classic RAG pipeline (embedding, retrieval, and generation) but more sophisticated components are inserted. In this pipeline, we have used the following add-ons:

- **Reranker**: This allows us to sort the context found during the retrieval step. This is one of the most widely used elements in advanced RAG because it has been seen to significantly improve results.
- **Query transformation**: In this case, we are using a simple query transformation. This is because we want to try to broaden our retrieval range, since some relevant documents may be missed.
- **Query routing**: This prevents us from treating all queries the same and allows us to establish rules for more efficient retrieval.
- **Hybrid search**: With this, we combine the power of keyword-based search with semantic search.
- **Summarization**: With this, we try to eliminate redundant information from our retrieved context.

Of course, we could add other components, but generally, these are the most commonly used and give an overview of what components we can add to naïve RAG.

We can see in the following figure how our pipeline is modified:

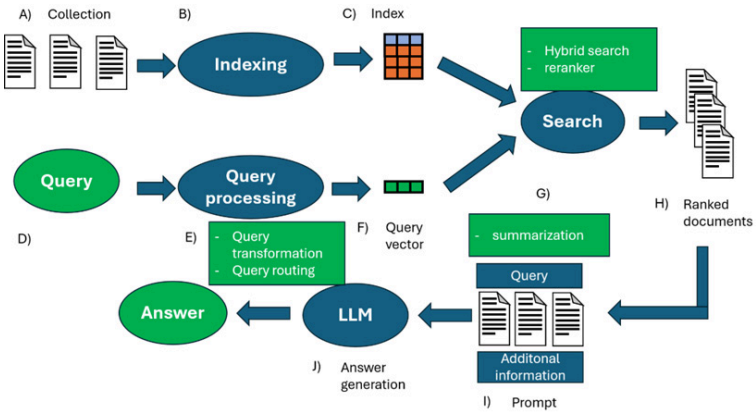

Figure 6.16 – Pipeline of advanced RAG

The complete code can be found in the repository; here, we will just see the highlights. In this code snippet, we are defining a function to represent the query transformation. In this case, we are developing only a small modification of the query (searching for other related terms in our query):

```
def advanced_query_transformation(query):
    """
    Transforms the input query by adding synonyms, extensions, or modifying the str
    for better search performance.
```

```
    Args:
        query (str): The original query.
    Returns:
        str: The transformed query with added synonyms or related terms.
    """
    expanded_query = query + " OR related_term"
    return expanded_query
```

Next, we perform query routing. Query routing enforces a simple rule: if specific keywords are present in the query, a keyword-based search is performed; otherwise, a semantic (embedding-based) search is used. In some cases, we may want to first retrieve only documents that contain certain keywords—such as references to a specific product—and then narrow the results further using semantic search:

```
def advanced_query_routing(query):
    """
    Determines the retrieval method based on the presence of specific keywords in t
    Args:
        query (str): The user's query.
    Returns:
        str: 'textual' if the query requires text-based retrieval, 'vector' otherwi
    """
    if "specific_keyword" in query:
        return "textual"
    else:
        return "vector"
```

Next, we perform a hybrid search, which allows us to use search based on semantic and keyword content. This is one of the most widely used components in RAG pipelines today. When chunking is used, sometimes documents relevant to a query can only be found because they contain a keyword (e.g., the name of a product, a person, and so on). Obviously, not all chunks that contain a keyword are relevant documents (especially for queries where we are more interested in a semantic concept). With hybrid search, we can balance the two types of search, choosing how many chunks to take from one or the other type of search:

```
def fusion_retrieval(query, top_k=5):
    """
    Retrieves the top_k most relevant documents using a combination of vector-based
    and textual retrieval methods.
    Args:
        query (str): The search query.
        top_k (int): The number of top documents to retrieve.
    Returns:
        list: A list of combined results from both vector and textual retrieval met
    """
    query_embedding = sentence_model.encode(query).tolist()
    vector_results = collection.query(query_embeddings=[query_embedding], n_results
    es_body = {
        "size": top_k,  # Move size into body
        "query": {
            "match": {
                "content": query
            }
        }
    }
    es_results = es.search(index=index_name, body=es_body)
    es_documents = [hit["_source"]["content"] for hit in es_results['hits']['hits']
```

```
    combined_results = vector_results['documents'][0] + es_documents
    return combined_results
```

As mentioned, the reranker is one of the most frequently used elements; it is a transformer that is used to reorder the context. If we have found 10 chunks, we reorder the found chunks and usually take a subset of them. Sometimes, semantic search can find the most relevant chunks again, but these may then be found further down the order. The reranker ensures that these chunks are then actually placed in the context of the LLM:

```
def rerank_documents(query, documents):
    """
    Reranks the retrieved documents based on their relevance to the query using a p
    BERT model.
    Args:
        query (str): The user's query.
        documents (list): A list of documents retrieved from the search.
    Returns:
        list: A list of reranked documents, sorted by relevance.
    """
    inputs = [rerank_tokenizer.encode_plus(query, doc, return_tensors='pt', truncat
    scores = []
    for input in inputs:
        outputs = rerank_model(**input)
        logits = outputs.logits
        probabilities = F.softmax(logits, dim=1)
        positive_class_probability = probabilities[:, 1].item()
        scores.append(positive_class_probability)
    ranked_docs = sorted(zip(documents, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, score in ranked_docs]
```

As mentioned earlier, context can also contain information that is redundant. LLMs are sensitive to noise, so reducing this noise can help generation. In this case, we use an LLM to summarize the found context (of course, we set a limit to avoid losing too much information):

```
def select_and_compress_context(documents):
    """
    Summarizes the content of the retrieved documents to create a compressed contex
    Args:
        documents (list): A list of documents to summarize.
    Returns:
        list: A list of summarized texts for each document.
    """
    summarized_context = []
    for doc in documents:
        input_length = len(doc.split())
        max_length = min(100, input_length)  than 100
        summary = summarizer(doc, max_length=max_length, min_length=5, do_sample=Fa
        summarized_context.append(summary)
    return summarized_context
```

Once defined, we just need to assemble them into a single pipeline. Once that's done, we can use our RAG pipeline. Check the code in the repository and play around with the code. Once you have a RAG pipeline that works, the next natural step is deployment. In the next section, we will discuss potential challenges to the deployment.

Understanding the scalability and performance of RAG

In this section, we will mainly describe challenges that are related to the commissioning of a RAG system or that may emerge with the scaling of the system. The main advantage of RAG over an LLM is that it can be scaled without conducting additional training. The purpose and requirements of development and production are mainly different. LLMs and RAG pose new challenges, especially when you want to take a system into production. Productionizing means taking a complex system such as RAG from a prototype to a stable, operational environment. This can be extremely complex when you have to manage different users who may be connected remotely. While in development, accuracy might be the most important metric, while in production, special care must be taken to balance performance and cost.

Large organizations, in particular, may already have big data stored and may therefore want to use RAG with it. Big data can be a significant challenge for a RAG system, especially considering the volume, velocity, and variety of data. **Scalability** is a critical concern when discussing big data; the same principle applies to RAG.

Data scalability, storage, and preprocessing

So far, we have talked about how to find information. We have assumed that the data is in textual form. The data structure of the text is an important parameter, and putting it into production can be problematic. So, our system may have to integrate the following:

- **Unstructured data**: Text is the most commonly used data type present in a corpus. It can have different origins: encyclopedic (from Wikipedia), domain-specific (scientific, medical, or financial), industry-specific (reports or standard documents), downloaded from the internet, or user chat. It can thus be generated by humans but also include data generated by automated systems or by LLMs themselves (previous interactions with users). In addition, it can be multi-language, and the system may have to conduct a cross-language search. Today, there are both LLMs that have been trained with different languages and multi-lingual embedders (specifically designed for multilingual capabilities). There are also other types of unstructured data, such as image and video. We will discuss multimodal RAG in a little more detail in the next section.
- **Semi-structured data**: Generally, this means data that contains a mixture of textual and table information (such as PDFs). Other examples of semi-structured data are JSON, XML, and HTML. These types of data are often complex to use with RAG. There are usually file-specific pipelines (chunking, metadata storing, and so on) because they can create problems for the system. In the case of PDF, chunking can separate tables into multiple chunks, making retrieval inefficient. In addition, tables make similarity search more complicated. An alternative is to extract the tables and turn them into text or insert them into compatible databases (such as SQL). Since the available methods are not yet optimal, there is still intense research in the field.
- **Structured data**: Structured data is data that is in a standardized format that can be accessed efficiently by both humans and software.

Structured data generally has some special features: defined attributes (same attributes for all data values as in a table), relational attributes (tables have common values that tie different datasets together; for example, in a customer dataset, there are IDs that allow users and their purchases to be found), quantitative data (data is optimized for mathematical analysis), and storage (data is stored in a particular format and with precise rules). Examples of structured data are Excel files, SQL databases, web form results, point-of-sale data, and product directories. Another example of structured data is KGs, which we will discuss in detail in the next chapter.

These factors must be taken into account. For example, if we are designing a system that needs to search for compliance documents in various regions and in different languages, we need a RAG that can conduct cross-lingual retrieval. If our organization has primarily one type of data (PDF or SQL databases), it is important to take this into account and optimize the system to search for this type of data. There are specific alternatives to improve the capabilities of RAGs with structured data. One example, chain-of-table, is a method that integrates CoT prompting with table transformations. In a step-by-step process with an LLM and a set of predefined operations, it extracts and modifies tables. This approach is designed for handling complex tables, and it exploits step-by-step reasoning and step-by-step tabular operations to accomplish this. This approach is useful if we have complex SQL databases or large amounts of data frames as data sources. Then, there are more sophisticated alternatives that combine symbolic reasoning and textual reasoning. Mix self-consistency is a dedicated approach to tabular data understanding that uses textual and symbolic reasoning with self-consistency, thus creating multi-paths of reasoning and then aggregating with self-consistency. For semi-structured data such as PDFs and JHTML, there are dedicated packages that allow us to extract information from them or to parse data.

It is not only the type of data that impacts RAG performance but also the amount of data itself. As the volume of data increases, so does the difficulty in finding relevant information. Likewise, it is likely to increase the latency of the system.

Data storage is one of the focal points to be addressed before bringing the system into production. Distributed storage systems (an infrastructure that divides data into several physical servers or data centers) can be a solution for large volumes of data. This has the advantage of increasing system speed and reducing the risk of data loss, but risks increasing costs and management complexity. When you have different types of data, it can be advantageous to use a structure called a data lake. A **data lake** is a centralized repository that is designed for the storage and processing of structured, semi-structured, and unstructured data. The advantage of the data lake is that it is a scalable and flexible structure for ingesting, processing, and storing data of different types. The data lake is advantageous for RAG because it allows more data context to be maintained than other data structures. On the other hand, data lakes require more expertise to be functional. Alternatives may be partitioning data into smaller, more manageable partitions (based on geography, topic, time, and so on), which allows more efficient retrieval. In the case of numerous requests, frequently accessed data caching can be conducted to avoid repetition. These strategies can be used in the case of big data storage and access.

Another important aspect is building solid pipelines for **data preprocessing and cleaning**. In the development stage, it is common to work with well-polished datasets, but in production, this is not the case. Especially in big data, it is essential to make sure that there are no inconsistencies or that the system can handle missing or incomplete data. In a big data environment, data comes from many sources and not all of them are good quality. Therefore, imputation techniques (KNN or others) can be used to fill in missing data. Other additions that can improve the process are techniques to eliminate noisy or erroneous data, such as outlier detection algorithms, normalization techniques, and regular expression techniques to eliminate erroneous data points.

Data deduplication is another important aspect when working with LLMs. Duplicate data harms the training of LLMs and is also detrimental when found during the generation process (risking outputs that are inaccurate, biased, or of poor quality). As the volume of data increases, data duplication is a risk that increases linearly. There are techniques such as fuzzy matching and hash-based deduplication that can be used to eliminate duplicate elements. In general, a pipeline should be created to control the quality and governance of the data in the system (data quality monitoring). These pipelines should include rules and tracking systems to be able to identify problematic data and its origin. Although these pipelines are essential, pipelines that are too complex to maintain or slow down the system too much should be avoided.

Once we have decided on our data storage infrastructure, we need to make sure we have efficient **data indexing and retrieval**. There are indexing methods that are specialized for big data, such as Apache Lucene or Elasticsearch. Also, the most used data can be cached, or the retrieval process can be distributed to create a parallel infrastructure and reduce bottlenecks when there are multiple users. Given the complexity of some of these techniques, it is always best to test and conduct benchmarks before putting them into production.

Parallel processing

Especially for applications with a large number of users, **parallel processing** can significantly increase system scalability. This obviously requires a good cloud infrastructure with well-organized clusters. Applying parallel processing to RAG significantly decreases system latency even when there are large datasets. Apache Spark and Dask are among the most widely used solutions for implementing parallel computing with RAG. As we have seen, parallel computing can be implemented at various stages of the RAG pipeline: storage, retrieval, and generation. During storage, the various nodes can be used to implement the entire data preprocessing pipeline, that is, preprocessing, indexing, and chunking of part of the dataset (up to embedding). Although it seems less intuitive, during retrieval, the dataset can be divided among various nodes, with each node responsible for finding information from a particular dataset shard. In this way, we reduce the computational burden on each node and make the retrieval process parallel.

Similarly, generation can be made parallel. In fact, LLMs are computationally intensive but are transformer-based. The transformer was designed with both parallelization of training and inference in mind. There are techniques that allow parallelization in the case of long sequences or

large batches of data. Later, more sophisticated techniques, such as tensor parallelism, model parallelism, and specialized frameworks, were developed. Paralleling the system, however, has inherent challenges and the risk of emerging errors. For these reasons, it is important to monitor the system during use and implement fault-tolerance mechanisms (such as checkpoints), advanced scheduling (such as dynamic task assignment), and other potential solutions.

RAG is a resource-intensive process (or at least some of the steps are), so it is good practice to implement techniques that dynamically allocate resources and monitor the workloads of the various processes. Also, it is recommended to use a modular approach that separates the various components, such as data ingestion, storage, retrieval, and generation. In any case, it is advisable to have a process that monitors not only performance in terms of accuracy but also memory usage, costs, network usage, and so on.

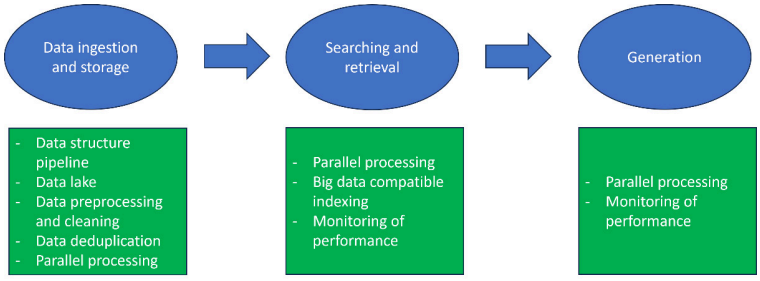

Figure 6.17 – Big data solutions for RAG scalability

We have talked generally about RAG. As we saw earlier, though, RAG today can be composed of several components. With advanced RAG and modular RAG, we saw how this system can be rapidly extended with additional components that impact both the accuracy of the system and its computational and latency costs. Thus, there are many alternatives for our system, and it is difficult to choose which components are most important. To date, there are a few benchmark studies that have conducted a rigorous analysis of both performance and computational costs. In a recent study (Wang, 2024), the authors analyzed the potential best components and gave guidance on which elements to use. In *Figure 6.18*, the components marked in blue are those, according to the authors of the study, that give the best performance, while those in bold are optional components.

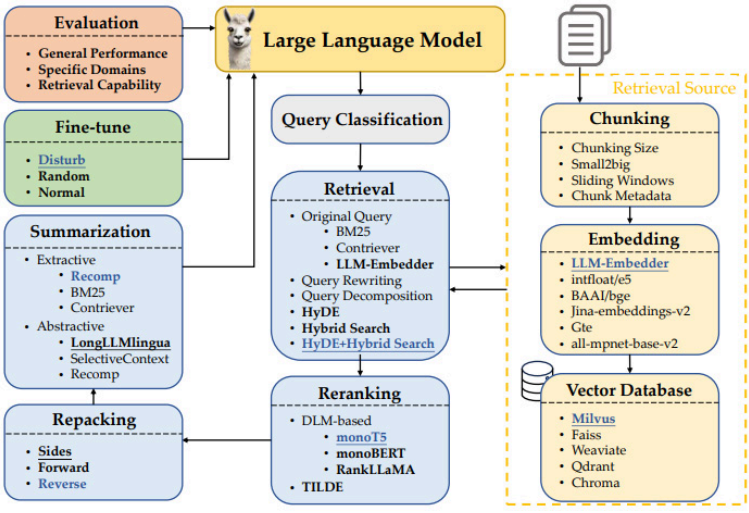

Figure 6.18 – Contribution of each component for an optimal RAG (https://arxiv.org/pdf/2407.01219)

For example, the addition of some components improves system accuracy with a noticeable increase in latency. HyDE achieves the highest performance score but seems to have a significant computational cost. In this case, the performance improvement does not justify this increased latency. Other components increase the computational cost, but their absence results in an appreciable drop in performance (this is the case with reranking). Summarization modules help the model achieve optimal accuracy; their cost can be justified if latency is not problematic. Although it is virtually impossible to test all components in a systematic search, some guidelines can be provided. The best performance is achieved with the query classification module, HyDE, the reranking module, context repacking, and summarization. If this is too expensive computationally or in terms of latency, however, it is better to avoid techniques such as HyDE and stick to the other modules (perhaps choosing less expensive alternatives, for example, a reranker with fewer parameters). This is summarized in the following table comparing individual modules and techniques in terms of performance and computational efficiency:

Method	Commonsense	Fact Check	ODQA		Multihop		Medical	RAG	Avg.		
	Acc	Acc	EM	F1	EM	F1	Acc	Score	Score	F1	Latency
classification module, Hybrid with HyDE, monoT5, sides, Recomp											
w/o classification	0.719	0.505	0.391	**0.450**	**0.212**	0.255	**0.528**	0.540	0.465	**0.353**	16.58
+ classification	**0.727**	**0.595**	**0.393**	**0.450**	0.207	**0.257**	0.460	**0.580**	**0.478**	**0.353**	**11.71**
with classification, *retrieval module*, monoT5, sides, Recomp											
+ HyDE	0.718	**0.595**	0.320	0.373	0.170	0.213	0.400	0.545	0.443	0.293	11.58
+ Original	0.721	0.585	0.300	0.350	0.153	0.197	0.390	0.486	0.428	0.273	**1.44**
+ Hybrid	0.718	**0.595**	0.347	0.397	0.190	0.240	**0.750**	0.498	0.477	0.318	1.45
+ Hybrid with HyDE	**0.727**	**0.595**	**0.393**	**0.450**	**0.207**	**0.257**	0.460	**0.580**	**0.478**	**0.353**	11.71
with classification, Hybrid with HyDE, *reranking module*, sides, Recomp											
w/o reranking	0.720	0.591	0.365	0.429	0.211	**0.260**	**0.512**	0.530	0.470	0.334	**10.31**
+ monoT5	**0.727**	0.595	0.393	0.450	0.207	0.257	0.460	**0.580**	**0.478**	0.353	11.71
+ monoBERT	0.723	0.593	0.383	0.443	**0.217**	0.259	0.482	0.551	0.475	0.351	11.65
+ RankLLaMA	0.723	**0.597**	0.382	0.443	0.197	0.240	0.454	0.558	0.470	0.342	13.51
+ TILDEv2	0.725	0.588	**0.394**	**0.456**	0.209	0.255	0.486	0.536	0.476	**0.355**	11.26
with classification, Hybrid with HyDE, monoT5, *repacking module*, Recomp											
+ sides	0.727	0.595	**0.393**	**0.450**	0.207	0.257	0.460	**0.580**	0.478	0.353	11.71
+ forward	0.722	**0.599**	0.379	0.437	0.215	0.260	0.472	0.542	0.474	0.349	**11.68**
+ reverse	**0.728**	0.592	0.387	0.445	**0.219**	**0.263**	**0.532**	0.560	**0.483**	**0.354**	11.70
with classification, Hybrid with HyDE, monoT5, reverse, *summarization module*											
w/o summarization	**0.729**	0.591	**0.402**	**0.457**	0.205	0.252	0.528	0.533	0.480	**0.355**	**10.97**
+ Recomp	0.728	**0.592**	0.387	0.445	**0.219**	**0.263**	**0.532**	**0.560**	**0.483**	0.354	11.70
+ LongLLMLingua	0.713	0.581	0.362	0.423	0.199	0.245	0.530	0.539	0.466	0.334	16.17

Figure 6.19 – Impact of single modules and techniques on accuracy and latency (https://arxiv.org/pdf/2407.01219)

In addition, there are also parallelization strategies specifically designed for RAG. LlamaIndex offers a parallel pipeline for data ingestion and processing. In addition, to increase the robustness of the system, there are systems to prevent errors. For example, when using a model, you may encounter runtime errors (especially if you use external APIs such as OpenAI or Anthropic). In these cases, it pays to have fallback models. An **LLM router** is a system that allows you to route queries to different LLMs. Typically, there is a predictor model to intelligently decide which LLM is best suited for a given prompt (taking into account potential accuracy or factors such as cost). These routers can be used either as closed source models or to route queries to different external LLM APIs.

Security and privacy

An important aspect to consider when a system goes into production is the **security and privacy** of the system. RAG can handle an enormous amount of sensitive and confidential data; breaching the system can lead to devastating consequences for an organization (regulatory fines, lawsuits, reputational damages, and so on). One of the main solutions is data encryption. Some algorithms and protocols are widely used in the indus-

try and can also be applied to RAG (e.g., AES-256 and TLS/SSL). Similarly, it is important to implement internal policies to safeguard keys and change them frequently. In addition, a system of credentials and privileges must be implemented to ensure controlled access by users. It is good practice today to use methods such as **multi-factor authentication (MFA)**, strong password rules, and policies for access from multiple devices. Again, an important part of this is continuous monitoring of potential breakage, incident reporting, and policies if they occur. Before deployment, it is essential to conduct testing of the system and its robustness to identify potential vulnerabilities.

Privacy is a crucial and increasingly sensitive topic today. It is important that the system complies with key regulations such as the **General Data Protection Regulation (GDPR)** and the **California Consumer Privacy Act (CCPA)**. Especially when handling large amounts of personal data, violations of these regulations expose an organization to hefty fines. To avoid penalties, it is a good idea to implement robust data governance, tracking practices, and data management. There are also techniques that can be used to improve system privacy, such as differential privacy and secure multi-party computation. In addition, incidents should be tracked and there should be policies for handling problems and resolving them.

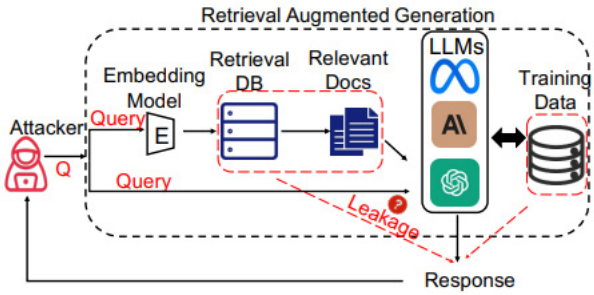

Figure 6.20 – The RAG system and potential risks (https://arxiv.org/pdf/2402.16893)

Then, there are several security problems today that are specific to RAG systems. For example, vectors might look like simple numbers but in fact can be converted back into text. The embedding process can be seen as lossy, but that doesn't mean it can't be decoded into the original text. In theory, embedding vectors should only maintain the semantic meaning of the original text, thus protecting sensitive data. In fact, in some studies, they have been able to recover more than 70% of the words in the original text. Moreover, extremely sophisticated techniques are not necessary. In what are called **embedding inversion attacks**, you acquire the vectors and then decode them into the original text. In other words, contrary to popular belief, you can reconstruct text from vectors, and so these vectors should be protected as well. In addition, any system that includes an LLM is susceptible to **prompt injection attacks**. This is a type of attack in what looks like a legitimate prompt where malicious instructions are added. This could be to prompt the model to leak information. Prompt injection is one of the greatest risks to models, and often, new methods are described in the literature, so all previous precautions quickly become obsolete. In addition, particular prompts can induce outputs that are not expected by RAG. Adversarial prefixes are prefixes added to what is a prompt for RAG and can induce the generation of hallucinations and factual incorrect outputs.

Another type of attack is **poisoning RAG**, in which an attempt is made to enter erroneous data that will then be used by the LLM to generate skewed outputs. For example, to generate misinformation, we can craft target text that when injected will cause the system to generate a desired output. In the example in the figure, we inject text to poison RAG to influence the answer to a question.

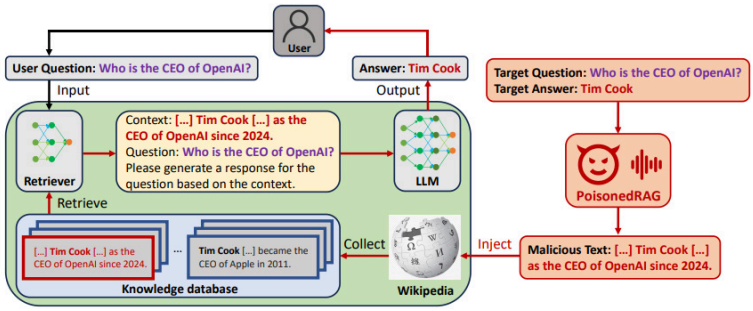

Figure 6.21 – Overview of poisoned RAG (https://arxiv.org/pdf/2402.07867)

Membership inference attacks (**MIAs**) are another type of attack in which an attempt is made to infer whether certain data is present within a dataset. If a sample resides in the RAG dataset, it will probably be found for a particular query and inserted into the context of an LLM. With an MIA, we can know if a piece of data is present in the system and then try to extract it with prompt injection (e.g., by making LLM output the retrieved context).

That is why there are specific solutions for the RAG (or for LLMs in general). One example is **NeMo Guardrails**, which is an open source toolkit developed by NVIDIA to add programmable rails to LLM-based applications. These rails provide a mechanism to control the LLM output of a model (so we act directly at the generation level). In this way, we can provide constraints (not engaging in harmful topics, following a path during dialog, not responding to certain requests, using a certain language, and so on). The advantage of this approach over other embedded techniques (such as model alignment at training) is that it happens at runtime and we do not have to conduct additional training for the model. This approach is also model agnostic and, generally, these rails are interpretable (during alignment, we should analyze the dataset used for training). NeMo Guardrails implements user-defined programmable rails via an interpretable language (called Colang) that allows us to define behavior rules for LLMs.

With this toolkit, we can use different types of guardrails: input rails (reject input, conduct further processing, or modify the input, to avoid leakage of sensitive information), output rails (refuse to produce outputs in case of problematic content), retrieval rails (reject chunks and thus do not put them in the context for LLM, or alter present chunks), or dialog rails (decide whether to perform an action, use the LLM for a next step, or use a default response).

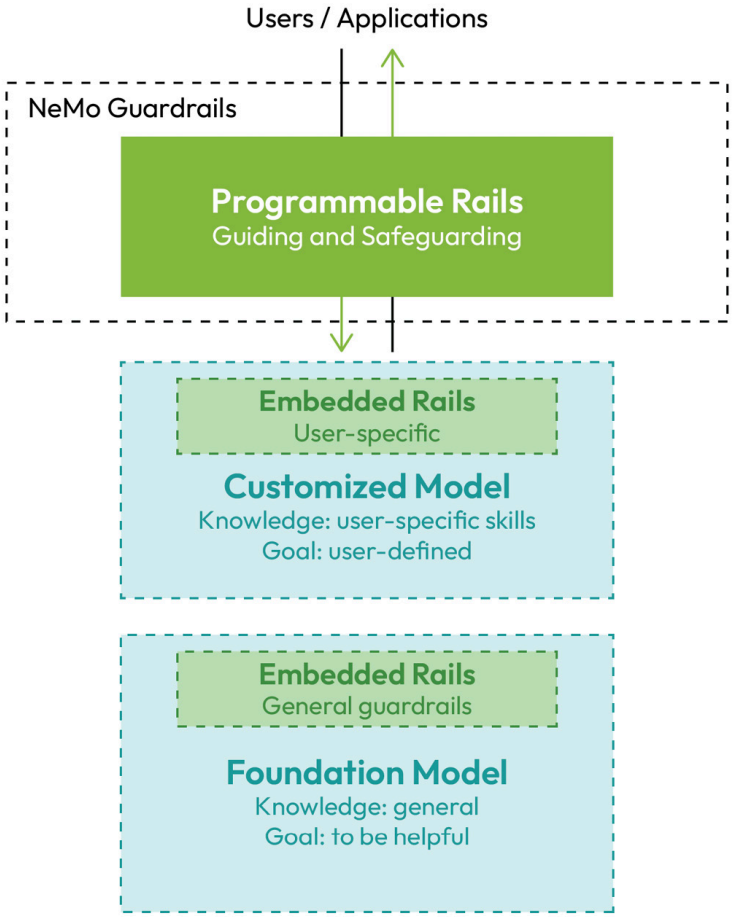

Figure 6.22 – Programmable versus embedded rails for LLMs (https://arxiv.org/abs/2310.10501)

Llama Guard, on the other hand, is a system designed to examine input (via prompt classification) and output (via response classification) and judge whether the text is safe or unsafe. This approach then uses Llama 2 for classification and then uses a specifically adapted LLM as the judge.

Open questions and future perspectives

Although there have been significant advances in RAG technology, there are still challenges. In this section, we will discuss these challenges and prospects.

Recently, there has been wide interest and discussion about the expansion of the context length of LLMs. Today, most of the best-performing LLMs have a context length of more than 100K tokens (some up to over 1 million). This capability means that a model has the capacity for long document question-answering (in other words, the ability to insert long documents such as books within a single prompt). Many small user cases can be covered by a context length of 1 to 10 million tokens. The advantage of a **long-context LLM (LC-LLM)** is that it can then conduct interleaved retrieval and generation of the information in the prompt and conduct one-shot reasoning over the entire document. Especially for summarization tasks, the LC-LLM has a competitive advantage because it can conduct a scan of the whole document and relate information present at the top and bottom of the document. For some, LC-LLM means that the RAG is doomed to disappear.

In reality, the LC-LLM does not compete with RAG, and RAG is not doomed to disappear in the short term. The LC-LLM does not use the whole framework efficiently. In particular, the information in the middle of the context is attended much less efficiently. Similarly, reasoning is impacted by irrelevant information, and a long prompt inevitably provides an unnecessary amount of detail to answer a query. The LC-LLM hallucinates much more than RAG, and the latter allows for reference checking (which documents were used, thus making the retrieval and reasoning process observable and transparent). The LC-LLM also has difficulty with structured data (which is most data in many industries) and has a fairly considerable cost (latency increases significantly with a long prompt and also the cost per query). Finally, 1 million tokens are not a lot when considering the amount of data that even a small organization has (so retrieval is always necessary).

The LC-LLM opens up exciting possibilities for developers. First, it means that a precise chunking strategy will be necessary much less frequently. Chunks can be much larger (up to a document per chunk or at least a group of pages). This will mean less need to balance granularity and performance. Second, less prompt engineering will be needed. Especially for reasoning tasks, some questions can be answered with the information in one chunk, but others require deep analysis among several sections or multiple documents. Instead of a complex CoT, it is possible to answer these questions with a single prompt. Third, summarization is easier with the LC-LLM, so it can be conducted with a single retrieval. Finally, the LC-LLM allows for better customization and interaction with the user. In such a long prompt, it will be possible to upload the entire chat with the user. There are still some open challenges, though, especially in retrieving documents for the LC-LLM.

Similarly, there are no embedding models today that can handle similar context lengths (currently, the maximum context length of an embedder is 32K). Therefore, even with an LC-LLM, the chunks cannot be larger than 32K. The LC-LLM is still expensive in terms of performance and can seriously impact the scalability of the system. In any case, there are already potential RAG variations being studied that take the LC-LLM into account – for example, adapting small-to-big retrieval in which you find the necessary chunks and then send the entire document associated with the LC-LLM, or conduct routing of a query to pipeline whole-document retrieval (such as whole-document summarization tasks) or to find chunks (specific questions or multi-part questions that require chunks of different documents). Many companies work with KV caching, which is an approach in which you store the activations from the key and query from an attention layer (so you don’t have to recompute the entire activations for a sequence during generation). So, it has been proposed that RAG could also be used to find the cache

We can see these possible evolutions visually in the following figure:

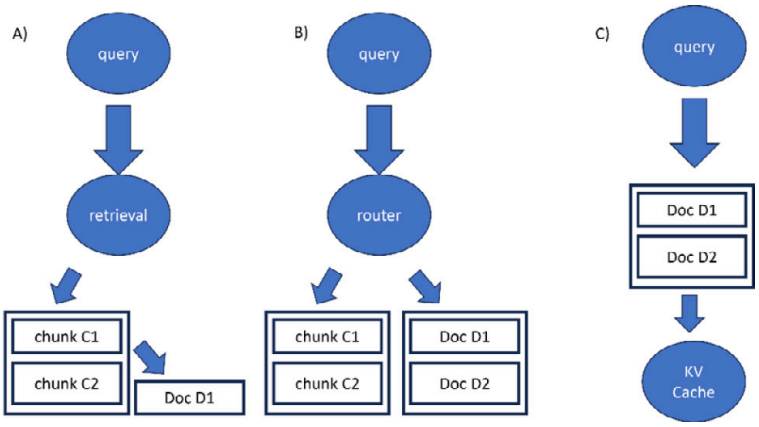

Figure 6.23 – Possible evolution of RAG with the LC-LLM

- A. Retrieving first the chunks and then the associated documents
- B. Router deciding whether it is necessary to retrieve small chunks or whole documents
- C. Retrieving the document and then KV caching them for the LC-LLM

Multimodal RAG is an exciting prospect and challenge that has been discussed. Most organizations have not only textual data but also extensive amounts of data in other modalities (images, audio, video, and so on). In addition, many files may contain more than one modality (for example, a book that contains not only text but also images). Searching for multimodal data can be of particular interest in different contexts and different applications. On the other hand, multimodal RAG is complicated by the fact that each modality has its own challenges. There are some alternatives to how we can achieve multimodal RAG. We will see three possible strategies:

- **Embed all modalities into the same vector space**: We previously saw the case of CLIP in ***Chapter 3*** (a model trained by contrastive learning to achieve unique embedding for images and text), which allowed us to search both images and text. We can use a model such as CLIP to conduct embedding of all modalities (in this case, images and text, but other cross-modal models exist). We can then find both images and text and use a multimodal model for generation (for example, we can use BLIP2 or BLIP3 as a vision language model). A multimodal model can conduct reasoning about both images and text. This approach has the advantage that we only need to change the embedding model to our system. In addition, a multimodal model can conduct reasoning by exploiting the information in both the image and the text. For example, if we have a PDF with tables, we can find the chunk of interest and the associated graphs. The model can use the information contained in both modalities to be able to answer the query more effectively. The disadvantage is that CLIP is an expensive model, and **multimodal LLMs** (**MMLLMs**) are more expensive than text-only LLMs. Also, we need to be sure that our embedding model is capable of capturing all the nuances of images and text.
- **Single-grounded modality**: Another option is to transform all modes into the primary mode (which can be different depending on the focus of the application). For example, we extract text from the PDF and create text descriptions for each of the images along with metadata (for audio, we can use a transcript). In some variants, we keep the images in storage. During retrieval, we find the text again (so we use a classic embedding model and a database that contains only vectors obtained from text). We can then use an LLM or MMLLM (if we want to add the

images obtained by retrieving metadata or description) during the generation phase. Again, the main advantage is that we do not have to train any new type of model, but it can be expensive as an approach, and we lose some nuances from the image.

- **Separate retrieval for each modality**: In this case, each modality is embedded separately. For example, if we have three modalities, we will have three separate models (audio-text-aligned model, image-text-aligned model, and text embedder) and three separate databases (audio, images, and text). When the query arrives, we encode for each mode (so audio, images, and text). So, in this case, we have done three retrievals and may have found different elements, so it pays to have a rerank step (to efficiently combine the results). Obviously, we need a dedicated multimodal rerank that can allow us to retrieve the most relevant chunks. It simplifies the organization because we have dedicated models for each mode (a model that works well for all modes is difficult to obtain) but it increases the complexity of the system.
  Similarly, while a classical reranker has to reorder n chunks, a multimodal reranker has the complexity of reordering m x n chunks (where m is the number of modes).
  Finally, once the multimodal chunks have been obtained, there may be alternatives; for example, we can use an MMLM to generate a response, and then this response needs to be integrated into the context for a final LLM. As we saw earlier, our RAG pipeline can be more sophisticated than naïve RAG. We can then combine all the elements we saw earlier into a single system.

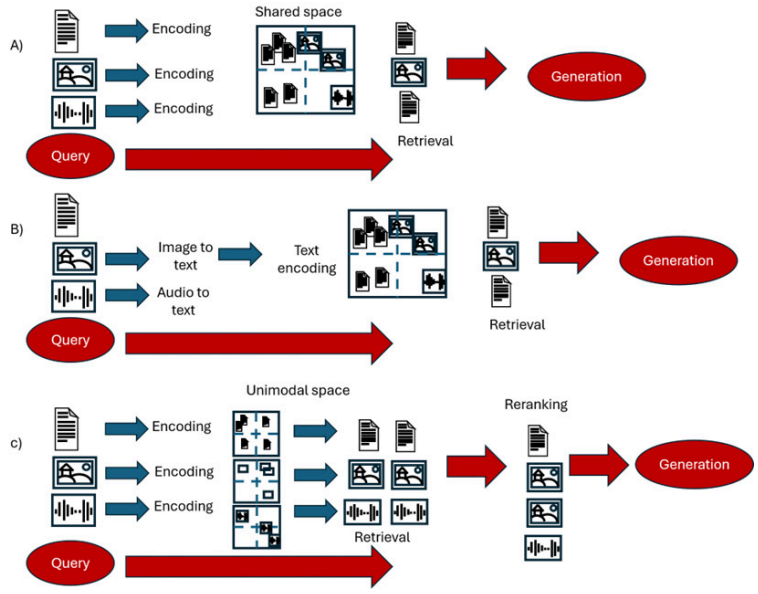

Figure 6.24 – Three potential approaches to multimodal RAG

Although RAG efficiently mitigates hallucinations, they can happen. We have previously discussed hallucinations as a plague of LLMs. In this section, we will mainly discuss hallucinations in RAG. One of the most peculiar cases is **contextual hallucinations**, in which the correct facts are provided in the context, but the LLM still generates the wrong output. Although the model provides the correct information, it produces a wrong answer (this often occurs in tasks such as summarization or document-based questions). This occurs because the LLM has its own prior knowledge, and it is wrong to assume that the model does not use this internal knowledge. Furthermore, the model is instruction-tuned or otherwise aligned, so it implicitly makes a decision on whether to use the context or ignore it and use its knowledge to answer the user's question. In

some cases, this might even be useful, since it could happen that we have found the wrong or misleading context. In general, for many closed source models, we do not know what they were trained on, though we can monitor their confidence in an answer. Given a question *x*, the model will respond with an answer *x*. Depending on its knowledge, this will have a confidence *c* (which is based on the probability associated with the tokens generated by the model). Basically, the more confident a model is in its answer, the less prone it will be to changing its answer if the context suggests differently. An interesting finding is that if the correct answer is slightly different from the LLM's knowledge, the LLM is likely to change its answer. In case of a large divergence, the LLM will choose its own answer. For example, to the question, "What is the maximum dosage of drug x?" the model may have seen 20 µg in its training. If the context suggests 30, the LLM will provide 30 as the output; if the context suggests 100, the LLM will state 20. Larger LLMs are generally more confident and prefer their answer, while smaller models are more willing to use context. Finally, this behavior can be altered with prompt engineering. Stricter prompts will force the model to use context, while weaker prompts will push the model to use its prior knowledge.

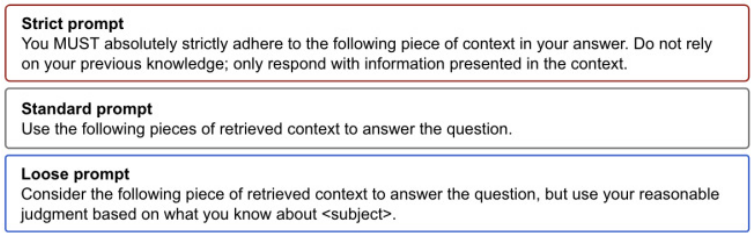

Figure 6.25 – Example of a standard prompt in comparison with a loose or strict prompt (**https://arxiv.org/pdf/2404.10198**)

Other factors also help reduce hallucinations in RAG:

- **Data quality**: Data quality has a big impact on system quality in general.
- **Contextual awareness**: The LLM may not best understand the user's intent, or the found context may not be the right one. Query rewriting and other components of advanced RAG might be the solution.
- **Negative rejection**: When retrieval fails to find the appropriate context for the query, the model attempts to respond anyway, thereby generating hallucinations or incorrect answers. This is often the fault of a poorly written query, so it can be improved with components that modify the query (such as HyDE). Alternatively, stricter prompts force the LLM to respond only if there is context.
- **Reasoning abilities**: Some queries may require reasoning or are too complex. The reasoning limit of the system depends on the LLM; RAG is for finding the context to answer the query.
- **Domain mismatch**: A generalist model will have difficulty with domains that are too technical. Fine-tuning the embedder and LLM can be a solution.
- **Objective mismatch**: The goals of the embedder and LLM are not aligned, so today there are systems that try to optimize end-to-end retrieval and generation. This can be a solution for complex queries or specialized domains.

There are other exciting perspectives. For example, there is some work on using reinforcement learning to improve the ability of RAG to respond to complex queries. Other research deals with integrating graph research;

we will discuss this in more detail in the next chapter. In addition, we have assumed so far that the database is static, but in the age of the internet, there is a discussion on how to integrate the internet into RAG (e.g., conducting a hybrid search in an organization's protected data and also finding context through an internet search). This opens up exciting but complex questions, such as whether or not to conduct database updates, how to filter out irrelevant search engine results, and security issues. In addition, there are more and more specialized applications of RAG, where the authors focus on creating systems optimized for their field of application (e.g., RAG for math, medicine, biology, and so on). All this shows active research into RAG and interest in its application.

Summary

In this chapter, we initially discussed what the problems of naïve RAG are. This allowed us to see a number of add-ons that can be used to solve the sore points of naïve RAG. Using these add-ons is the basis of what is now called the advanced RAG paradigm. Over time, the community then moved toward a more flexible and modular structure that is now called modular RAG.

We then saw how to scale this structure in the presence of big data. Like any LLM-based application, there are computational and cost challenges when you have to take the system from a development environment to a production environment. In addition, both LLMs and RAGs can have security and privacy risks. These are important points, especially when these products are open to the public. Today, there is an increasing focus on compliance and more and more regulations are being considered.

Finally, we saw that some issues remain open, such as the relationship with long-context LLMs or the multimodal extension of these models. In addition, there is a delicate balance between retrieval and generation, and we explored potential solutions in case of problems. Recently, there has been active research into integration with KGs. GraphRAG is often discussed today; in the next chapter, we will discuss what a KG is and the relationship between graphs and RAG.

Further reading

- LlamaIndex, *Node Postprocessor Modules*: **https://docs.llamaindex.ai/en/stable/module_guides/querying/node_postprocessors/node_postprocessors/**
- Nelson, *Lost in the Middle: How Language Models Use Long Contexts*, 2023: **https://arxiv.org/abs/2307.03172**
- Jerry Liu, *Unifying LLM-powered QA Techniques with Routing Abstractions*, 2023: **https://betterprogramming.pub/unifying-llm-powered-qa-techniques-with-routing-abstractions-438e2499a0d0**
- Chevalier, *Adapting Language Models to Compress Contexts*, 2023: **https://arxiv.org/abs/2305.14788**
- Li, *Unlocking Context Constraints of LLMs: Enhancing Context Efficiency of LLMs with Self-Information-Based Content Filtering*, 2023: **https://arxiv.org/abs/2304.12102**
- Izacard, *Distilling Knowledge from Reader to Retriever for Question Answering*, 2020: **https://arxiv.org/abs/2012.04584**

- Wang, *Searching for Best Practices in Retrieval-Augmented Generation*, 2024: **https://arxiv.org/pdf/2407.01219**
- Li, *Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach*, 2024: **https://www.arxiv.org/abs/2407.16833**
- Raieli, *RAG is Dead, Long Live RAG*, 2024: **https://levelup.gitconnected.com/rag-is-dead-long-live-rag-c607e1799199**
- Raieli, *War and Peace: A Conflictual Love Between the LLM and RAG*, 2024: **https://ai.plainenglish.io/war-and-peace-a-conflictual-love-between-the-llm-and-rag-78428a5776fb**
- jinaai/jina-colbert-v2: **https://huggingface.co/jinaai/jina-colbert-v2**
- **`mix_self_consistency`**: **https://github.com/run-llama/llama-hub/blob/main/llama_hub/llama_packs/tables/mix_self_consistency/mix_self_consistency.ipynb**
- Zeng, *The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)*, 2024: **https://arxiv.org/abs/2402.16893**
- Xue, *BadRAG: Identifying Vulnerabilities in Retrieval Augmented Generation of Large Language Models*, 2024: **https://arxiv.org/abs/2406.00083**
- Chen, *Controlling Risk of Retrieval-augmented Generation: A Counterfactual Prompting Framework*, 2024: **https://arxiv.org/abs/2409.16146**
- Zhang, *HijackRAG: Hijacking Attacks against Retrieval-Augmented Large Language Models*, 2024: **https://arxiv.org/abs/2410.22832**
- Xian, *On the Vulnerability of Applying Retrieval-Augmented Generation within Knowledge-Intensive Application Domains*, 2024: **https://arxiv.org/abs/2409.17275v1**

Table of contents collapsed